

TodaRide

Setup Guide

How to enable real Google Maps geocoding/routing, real OTP SMS delivery through a third-party gateway, and what it costs to keep the app running. Three independent sections — read them in any order.

Part 1

Google
Maps API
— accurate
barangay-
level
geocoding
& road
routing

Part 2

Real OTP
SMS —
texting a
real
verification
code via
Semaphore

Part 3

Ongoing
operating
budget —
what it
costs to
keep this
running

Both Part 1 and Part 2 features are already built into the app and currently running in free/simulated fallback mode — nothing here requires further coding. Each is a one-time account setup on the provider's website, followed by pasting a credential into a config file in the project. Part 3 is a budget reference, not a setup step.

Part 1 — Google Maps API

What this fixes: the app currently geocodes addresses and draws live-trip road routes using OpenStreetMap's free Nominatim/OSRM services. That free data has very few individually-mapped points for small barangays in areas like San Jose City, Nueva Ecija — so many different barangays resolve to the same city-center coordinate. Switching to Google's Geocoding and Directions APIs gives each barangay its own real, distinct location and traffic-aware road routing.

What's already built: the app automatically prefers Google's APIs the moment a key is present, and falls back to the free OSM stack if the key is missing or fails. No app changes are needed after setup — only the steps below.

Step-by-step

1. Go to console.cloud.google.com and sign in with a Google account.
2. Click the project dropdown (top left) -> New Project. Name it anything, e.g. "TodaRide" -> Create.
3. Enable billing: left menu -> Billing -> link a billing account (a card is required). This is required even to stay within Google's free monthly usage tier (~\$200/month free credit — comfortably enough for a prototype).
4. Enable the two required APIs: left menu -> APIs & Services -> Library. Search for "Geocoding API" -> open it -> Enable. Repeat for "Directions API".
5. Create a key: APIs & Services -> Credentials -> Create Credentials -> API key. Google generates the key immediately — copy it.
6. Restrict the key so it can't be abused if leaked: click the key -> under "API restrictions" choose "Restrict key" and check only Geocoding API and Directions API. Under "Application restrictions", leave as "None" while testing locally; once this app is deployed to a real domain, switch to "HTTP referrers" and list that domain.
7. In the project folder, copy `.env.example` to a new file named `.env.local`, then set:

```
VITE_GOOGLE_MAPS_API_KEY=paste_your_key_here
```

8. Restart the dev server (Vite only reads env vars at startup) — stop it if running, then run:

```
npm run dev
```

9. Done. Book a test ride in the app — pickup/dropoff pins should now land on distinct, real coordinates instead of a shared fallback point.

Note: `.env.local` is already listed in `.gitignore`, so the key is never committed if this project is later put under version control. Never share this key or paste it into chat, docs, or public repos.

Part 2 — Real OTP SMS (via Semaphore)

What this fixes: registration and login currently show the OTP code on-screen as a simulated "text message" because no SMS gateway is connected yet. The app already has a small local server (`server/index.js`) built to send real SMS through Semaphore — a Philippines-focused SMS gateway that supports local PH mobile numbers well. It just needs an account and an API key.

Why a server is involved at all: the SMS provider's API key is a paid secret — it cannot be placed directly in the browser app, because anyone opening dev tools could steal it and send SMS on your account's bill. The local server holds the key privately and the browser app just calls that server.

Step-by-step

1. Go to semaphore.co and click Sign Up. Register with an email and verify it.
2. Load credits (this is Semaphore's version of a "subscription" — it's prepaid, pay-as-you-go per SMS rather than a fixed monthly plan). From the dashboard, go to Billing / Load Credits and purchase a starter credit pack. Pricing is roughly PHP 0.50–PHP 1.00 per SMS depending on volume purchased (check current rates on their site, as these change over time).
3. Get your API key: dashboard -> Account -> API Key (sometimes labeled "API Keys" in the left sidebar). Copy it.
4. (Optional) Register a custom Sender Name: by default, messages show the sender as "SEMAPHORE". To show "TodaRide" as the sender instead, go to Sender Names in the dashboard and submit one for approval — this can take a few business days. Skip this step to keep the default sender and go live immediately.
5. In the project, open the server folder and copy `.env.example` to a new file named `.env` (inside that same server folder). Fill it in:

```
PORT=4000
SEMAPHORE_API_KEY=paste_your_semaphore_api_key_here
SEMAPHORE_SENDER_NAME=SEMAPHORE
```

Note: Change `SEMAPHORE_SENDER_NAME` to your approved custom name once step 4 is approved (max 11 characters, no spaces, per Semaphore's rules).

6. Install dependencies if this hasn't been done yet, then start the OTP server (run this in a separate terminal — it needs to stay running alongside the app):

```
npm install
npm run server
```

7. Confirm it started correctly — the terminal should print:

```
OTP server listening on http://localhost:4000
```

If instead it prints `"SEMAPHORE_API_KEY not set — sending will be simulated..."`, the key wasn't picked up — double-check the file is named exactly `server/.env` (not `.env.example`) and restart the server.

8. In a second terminal, start the app as usual:

```
npm run dev
```

9. Test end-to-end: open the app -> Sign up (Passenger, Parent, or Driver) -> enter a real Philippine mobile number -> Send OTP. You should receive an actual SMS on that phone within a few seconds, instead of the on-screen "Simulated SMS" message. Enter the code you received to verify.

Note: If the OTP server (step 6) isn't running, the app automatically falls back to showing the code on-screen — this is intentional, so the app stays fully testable without SMS credits. Once the server is running with a valid key, real texts go out automatically with no further app changes.

Part 3 — Ongoing Operating Budget

Current architecture, in plain terms: today the app is a client-only prototype — everything (rides, accounts, driver locations) lives in each device's browser storage. There is no shared server or database that syncs a passenger's phone with a driver's phone in real time. That keeps current running costs very low, but it also means the two paid integrations above (Parts 1 & 2) are the entire current operating cost — there is no hosting bill yet for the core app logic itself, since it can run as static files.

Monthly budget — as currently built

Note: Estimated for a small pilot: one TODA terminal, roughly 30–50 completed rides/day, with registration + occasional OTP logins.

Item	Est. monthly cost	Notes
Google Maps (Geocoding + Directions)	PHP 0	Pilot-scale call volume stays inside Google's ~\$200/month free credit.
Semaphore SMS (OTP texts)	PHP 25 – PHP 150	Prepaid credits; ~50–150 OTP texts/month at PHP 0.50–PHP 1.00 each.
Frontend hosting (static site)	PHP 0	Free tier on Vercel/Netlify/Cloudflare Pages comfortably covers a single-page app like this.
OTP server hosting (Node process)	PHP 280 – PHP 400 (~\$5–\$7)	Needs an always-on host, not a free tier that sleeps — a small Render/Railway instance or a basic VPS.
Domain name	PHP 55 – PHP 70 (~\$1–\$1.25)	Typical .com/.app registration, billed yearly (~\$12–15/yr) but shown here per month.
SSL certificate	PHP 0	Included free with virtually every host above (Let's Encrypt).
TOTAL (typical pilot)	~ PHP 360 – PHP 620 / month	Roughly \$6–\$11/month — dominated by the OTP server hosting, not the API usage itself.

If ride volume grows (~500 rides/day)

Item	Est. monthly cost	Notes
Google Maps (Geocoding + Directions)	PHP 1,100 – PHP 2,800 (~\$20–\$50)	Call volume at this scale likely exceeds the free credit; billed at roughly \$5 per 1,000 requests per API beyond it.
Semaphore SMS (OTP texts)	PHP 500 – PHP 1,500	More registrations and OTP logins per day.

OTP server hosting	PHP 550 – PHP 1,100 (~\$10–\$20)	Likely needs a slightly larger instance to stay responsive under more concurrent requests.
Frontend + domain + SSL	PHP 55 – PHP 70 (~\$1–\$1.25)	Same as pilot tier — these don't scale with ride volume.
TOTAL (growing pilot)	~ PHP 2,200 – PHP 5,500 / month	Roughly \$40–\$100/month.

Not included above — a real multi-user backend

The figures above keep the app exactly as it's built today: one device sees only its own browser data. Turning this into a real service where a passenger and driver on two different phones see the same live ride requires adding a shared backend — a real database plus a server that pushes live updates to both sides. That's a separate, larger engineering effort, not a config change, so no line-item number is invented for it here. As a rough category guide once that's built:

- Database + backend hosting (e.g. managed Postgres + a server host): commonly PHP 550–PHP 2,800/month (~\$10–\$50) at small-to-medium scale.
- Real-time sync infrastructure (WebSockets or a service like Firebase/Supabase Realtime): often bundled with the above, or a modest add-on cost.
- Payment gateway fees (GCash/Maya/card), once payments are real instead of simulated: typically 2%–3.5% per transaction, charged by the gateway, not a fixed monthly fee.

Figures are estimates based on published provider pricing at the time of writing; confirm current rates on each provider's site before budgeting precisely. Prototype · Simulated data — TodaRide setup reference.